



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

**Отчет по дисциплине «Промышленное программирование»
по лабораторной работе №1**

**Тема: «Разработка и интеграция модулей промышленного ПО на C/C++
+ с использованием CMake»**

**Проверил
преподаватель**

Мельников В.П.

подпись

**Выполнила
студентка группы ИДБ-25-06**

Якунина В.С.

подпись

Москва, 2026г

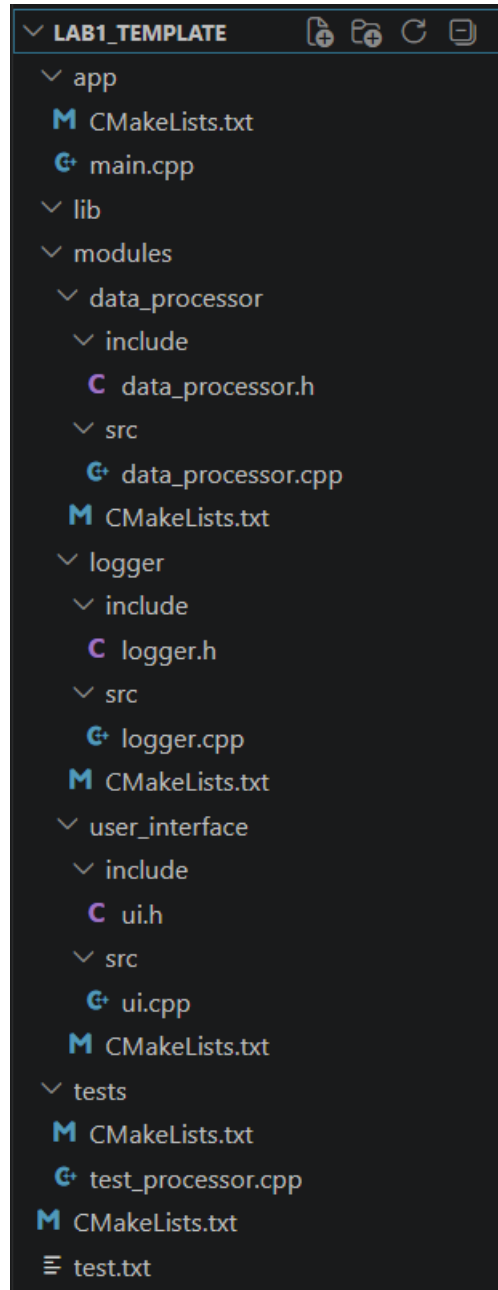
Оглавление

1 Цель работы.....	3
2 Структура проекта.....	4
3 Изменения.....	5
3.1 data_processor.cpp.....	5
3.2 logger.cpp.....	7
3.3 ui.cpp.....	9
3.4 test_processor.cpp.....	11
3.5 CmakeLists.txt.....	13
3.6 main.cpp.....	14
4 Тестирование.....	15
4.1 Google Test.....	15
4.2 Тест с ручным вводом анализируемого файла.....	16
4.2.1 С выводом сообщений о работе в консоль.....	16
4.2.2 С выводом сообщений о работе в файл для логов.....	16
5 Вывод.....	17

1 Цель работы

Закрепить на практике навыки модульного проектирования, получить опыт разработки независимых программных модулей (статических библиотек), их интеграции в единое приложение с помощью системы сборки CMake, а также проведения модульного и интеграционного тестирования.

2 Структура проекта



3 Изменения

3.1 data_processor.cpp

```
#include "data_processor.h"
#include <iostream>

ProcessResult process_line(const std::string& line) {
    // TODO: реализовать подсчёт слов, символов и поиск самого длинного слова

    int word_count = 0, char_count = 0;

    if (line[0] == '\\0')
    {
        return ProcessResult{ 0, 0, "" };
    }

    int max_len_word = 0, temp_len_word = 0;
    std::string temp_word;
    std::string long_word;
    for (int i = 0; i < line.length(); i++)
    {
        char_count++;
        if (line[i] == ' ' || line[i] == '\\n' || line[i] == '\\0')
        {
            if (temp_len_word > 0)
            {
                word_count++;
                if (temp_len_word > max_len_word)
                {
                    long_word = temp_word;
                    max_len_word = temp_len_word;
                }
                temp_word = "";
                temp_len_word = 0;
            }
        }
        else
        {
            temp_word += line[i];
            temp_len_word++;
        }
    }
}
```

```

    if (temp_len_word > 0) {
        word_count++;
        if (temp_len_word > max_len_word) {
            long_word = temp_word;
        }
    }

    // std::cout << "len word " << word_count << '\n' << "char_count "
    << char_count << '\n' << "longest word " << long_word << std::endl;

    return ProcessResult{ word_count, char_count, long_word };

    //return ProcessResult{0, 0, ""};
}

```

Программа принимает строку и возвращает количество слов в ней, количество символов и самое длинное слово.

3.2 logger.cpp

```
#include "logger.h"
#include <iostream>
#include <ctime>
#include <fstream>

bool is_open_file = false;
std::ofstream log_file;

void log_init(const std::string& filename) {
    // TODO: если filename не пуст, открыть файл для записи
    // иначе настроить вывод в консоль
    std::string message;
    if (filename[0] == '\\0')
    {
        log_message(INFO, "No file for logging. Logging to console");
        is_open_file = false;
    }
    else
    {
        //std::ifstream file(filename);

        log_file.open(filename, std::ios::out | std::ios::app);
        if (log_file.is_open())
        {
            is_open_file = true;
            log_message(INFO, "Logging to file");
        }
        else
        {
            log_message(ERROR, "No open" + filename);
        }
    }
}

void log_message(LogLevel level, const std::string& message) {
    // TODO: сформировать строку с временной меткой и уровнем,
    // записать в файл или в консоль

    std::string level_str;
    switch (level)
    {
        case LogLevel::INFO:
            level_str = "INFO";
            break;
        case LogLevel::ERROR:
            level_str = "ERROR";
```

```

        break;
    case LogLevel::WARNING:
        level_str = "WARNING";
        break;
    default:
        level_str = "UNKNOWN";
        break;
}

std::time_t now = std::time(nullptr);
std::tm* t = std::localtime(&now);
char time_buf[20];
std::strftime(time_buf, sizeof(time_buf), "%Y-%m-%d %H:%M:%S", t);

if (is_open_file)
{
    log_file << time_buf << ": " << level_str << ": ' ' << message <<
std::endl;
}
else
{
    std::cout << time_buf << " " << level_str << " ' ' << message <<
std::endl;
}
}

```

Программа принимает создает файл и записывает информацию о работе программы и ошибки в него, если название было дано при запуске программы, если нет — все записи ведутся в консоль.

3.3 ui.cpp

```
#include "ui.h"
#include "data_processor.h"
#include "logger.h"
#include <fstream>
#include <iostream>

void run_analysis(const std::string& filepath) {
    // TODO: открыть файл, построчно читать,
    // вызывать process_line, логировать результаты

    std::string message;

    std::ifstream file(filepath);
    if (!file.is_open())
    {
        log_message(ERROR, "No open " + filepath);
        return;
    }

    log_message(INFO, "Start analys");

    std::string line;
    int num_line = 0;
    while (std::getline(file, line))
    {
        ProcessResult result = process_line(line);
        num_line++;

        std::string message_results = "Results " +
std::to_string(num_line) + " string: \tWords : " +
        std::to_string(result.word_count) + "\tChars : " +
std::to_string(result.char_count) +
        "\tLongest word: ";

        if (!result.longest_word.empty())
        {
            message_results += result.longest_word + "\n";
        }
        else
        {
            message_results += "None\n";
        }
        std::cout << message_results << std::endl;
    }

    log_message(INFO, "Finish analys");
}
```

Программа осуществляет взаимодействие с пользователем: просит ввести файл для анализа и выводит сообщения об ошибках или информацию о работе программы.

3.4 test_processor.cpp

```
#include <gtest/gtest.h>
#include "data_processor.h"

// Пример теста (студенты должны дописать свои тесты)
TEST(ProcessorTest, EmptyLine) {
    ProcessResult res = process_line("");
    EXPECT_EQ(res.word_count, 0);
    EXPECT_EQ(res.char_count, 0);
    EXPECT_TRUE(res.longest_word.empty());
}

// TODO: добавить тесты для строк с одним словом,
// с несколькими словами, с ведущими/конечными пробелами и т.д.

TEST(ProcessorTest, OneWord) {
    ProcessResult res = process_line("W0000000ORD");
    EXPECT_EQ(res.word_count, 1);
    EXPECT_EQ(res.char_count, 10);
    EXPECT_EQ(res.longest_word, "W0000000ORD");
}

TEST(TestDataProcessor, AnyWord) {
    ProcessResult result = process_line("Hello big world");
    EXPECT_EQ(result.word_count, 3);
    EXPECT_EQ(result.char_count, 15);
    EXPECT_EQ(result.longest_word, "Hello");
}

TEST(TestDataProcessor, FirstProbel) {
    ProcessResult result = process_line("  Probel1 Probel54");
    EXPECT_EQ(result.word_count, 2);
    EXPECT_EQ(result.char_count, 18);
    EXPECT_EQ(result.longest_word, "Probel54");
}

TEST(TestDataProcessor, ProbelEnd) {
    ProcessResult result = process_line("Probel100 Probel3    ");
    EXPECT_EQ(result.word_count, 2);
    EXPECT_EQ(result.char_count, 23);
    EXPECT_EQ(result.longest_word, "Probel100");
}

TEST(TestDataProcessor, Probels) {
    ProcessResult result = process_line("        ");
    EXPECT_EQ(result.word_count, 0);
    EXPECT_EQ(result.char_count, 6);
    EXPECT_EQ(result.longest_word, "");
}
```

```

}

TEST(TestDataProcessor, VeryLongWords256) {
    ProcessResult result = process_line("hhhhheeeeeelllllloooooooo
pyyyyyyythooooooooooooon woooooooooooooorlddddddddddd hiiiiiiiiiii
aaaaaaaaaaaaaaaaaaaaa ooooooooooooooooooooo 5454545454545454545454545454
proooooooooooooograaaaaaaaaaaaaaaaaamiiiiiiiiiiiiiiiiingggggggggggggg
ooooooooooooooooooooonnnnnnnnnnnnn C++++++++++++++++++++");
    EXPECT_EQ(result.word_count, 10);
    EXPECT_EQ(result.char_count, 294);
    EXPECT_EQ(result.longest_word,
"proooooooooooooograaaaaaaaaaaaaaaaaamiiiiiiiiiiiiiiiiingggggggggggggg");
}

```

Программа тестирует работу анализа строк в файлах.

3.5 CmakeLists.txt

```
cmake_minimum_required(VERSION 3.15)

if(MSVC)
    set(CMAKE_MSVC_RUNTIME_LIBRARY
        "MultiThreaded$<$<CONFIG:Debug>:Debug>" CACHE STRING "" FORCE)
endif()

project(TextAnalyzer)

set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_subdirectory(modules/data_processor)
add_subdirectory(modules/logger)
add_subdirectory(modules/user_interface)
add_subdirectory(app)
add_subdirectory(tests)
```

Программа собирает проект, а также синхронизирует рантайм-библиотеки.

3.6 main.cpp

```
#include "ui.h"
#include "logger.h"
#include <iostream>
#include <string>

int main(int argc, char* argv[]) {
    if (argc > 1) {
        // первый аргумент - файл для логов
        log_init(argv[1]);
    } else {
        // без аргументов - консоль
        log_init();
    }

    std::string path;
    //std::cout << "Введите путь к текстовому файлу: ";
    std::cout << "Enter the path to the test file: ";
    std::getline(std::cin, path);

    if (path.empty()) {
        //log_message(ERROR, "Путь не указан");
        log_message(ERROR, "Path not specified");
        return 1;
    }

    run_analysis(path);
    return 0;
}
```

Main — главная функция, собирающая всю логику программы вместе: принимает файл для вывода в него сообщений и файл для анализа его строк.

4 Тестирование

4.1 Google Test

```
[=====] Running 7 tests from 2 test suites.
[-----] Global test environment set-up.
[-----] 2 tests from ProcessorTest
[ RUN    ] ProcessorTest.EmptyLine
[       OK ] ProcessorTest.EmptyLine (0 ms)
[ RUN    ] ProcessorTest.OneWord
[       OK ] ProcessorTest.OneWord (0 ms)
[-----] 2 tests from ProcessorTest (1 ms total)

[-----] 5 tests from TestDataProcessor
[ RUN    ] TestDataProcessor.AnyWord
[       OK ] TestDataProcessor.AnyWord (0 ms)
[ RUN    ] TestDataProcessor.FirstProbel
[       OK ] TestDataProcessor.FirstProbel (0 ms)
[ RUN    ] TestDataProcessor.ProbelEnd
[       OK ] TestDataProcessor.ProbelEnd (0 ms)
[ RUN    ] TestDataProcessor.Probels
[       OK ] TestDataProcessor.Probels (0 ms)
[ RUN    ] TestDataProcessor.VeryLongWords256
[       OK ] TestDataProcessor.VeryLongWords256 (0 ms)
[-----] 5 tests from TestDataProcessor (3 ms total)

[-----] Global test environment tear-down
[=====] 7 tests from 2 test suites ran. (6 ms total)
[ PASSED ] 7 tests.
```

4.2 Тест с ручным вводом анализируемого файла

Содержимое тестового файла:

Python Hello Hi World

Student Teacher School University

Fifty four 67 C++ long_word

4.2.1 С выводом сообщений о работе в консоль

```
2026-05-14 20:02:26 INFO No file for logging. Logging to console
Enter the path to the test file: C:\Users\neval\OneDrive\Desktop\PromProg\LAB1_TEMPLATE\test.txt
2026-05-14 20:02:53 INFO Start analys
Results 1 string:      Words : 4      Chars : 21      Longest word: Python
Results 2 string:      Words : 4      Chars : 33      Longest word: University
Results 3 string:      Words : 5      Chars : 27      Longest word: long_word
2026-05-14 20:02:53 INFO Finish analys
```

4.2.2 С выводом сообщений о работе в файл для логов

```
Enter the path to the test file: C:\Users\neval\OneDrive\Desktop\PromProg\LAB1_TEMPLATE\test.txt
Results 1 string:      Words : 4      Chars : 21      Longest word: Python
Results 2 string:      Words : 4      Chars : 33      Longest word: University
Results 3 string:      Words : 5      Chars : 27      Longest word: long_word
```

Содержимое файла для логов:

2026-05-14 20:07:23: INFO14880Logging to file

2026-05-14 20:07:33: INFO14880Start analys

2026-05-14 20:07:33: INFO14880Finish analys

5 Вывод

Были освоены навыки работы с snake и Google Test.